

VEKTORLABS

Frontend Blueprint

Complete Production Specification — Layout · Components · Data Flow · Security · Performance · Accessibility

React + Vite · WebSocket · JWT Auth · vektorlabs.xyz

Version	1.0 — Production Frontend Spec
Stack	React 18 + Vite + TailwindCSS + react-grid-layout
Audience	Semi-pro crypto futures traders
Route	vektorlabs.xyz/dashboard/{username}

Table of Contents

1.	Tech Stack & Project Structure
2.	Routing Architecture
3.	Page: Landing
4.	Page: Auth — Login + Register
5.	Page: Dashboard — Trading Terminal
6.	Page: Settings
7.	Page: Billing
8.	Page: Docs
9.	WebSocket Integration
10.	Component Specifications
11.	State Management
12.	Security
13.	Performance
14.	Accessibility
15.	Error Handling
16.	Testing Strategy
17.	Deployment

1. Tech Stack & Project Structure

Every dependency chosen for a reason. No bloat. If it is not in this list, do not add it without a documented reason.

Dependencies

Package	Version	Purpose
react	18.x	UI runtime
react-dom	18.x	DOM rendering
vite	5.x	Build tool — fast HMR, ESM
react-router-dom	6.x	Client-side routing
tailwindcss	3.x	Utility-first styling
react-grid-layout	1.x	Draggable/resizable widget grid
recharts	2.x	Charts — OFI, VPIN sparklines
zustand	4.x	Lightweight global state
@tanstack/react-query	5.x	Server state, caching, refetch
axios	1.x	HTTP client with interceptors
react-hot-toast	2.x	Non-blocking notifications
date-fns	3.x	Timestamp formatting
clsx	2.x	Conditional classNames

Project Structure

```
frontend/Vektor_Labs/src/ ■■■ components/ ■■■■ dashboard/ ■ ■■■■ widgets/ ■ ■ ■■■■
RegimeWidget.jsx ■ ■ ■■■■ MicrostructureWidget.jsx ■ ■ ■■■■ AltDataWidget.jsx ■ ■ ■■■■
LLMInsightWidget.jsx ■ ■ ■■■■ OrderBookWidget.jsx ■ ■ ■■■■ SummaryStrip.jsx ■ ■■■■
SymbolTabs.jsx ■ ■■■■ AlertStrip.jsx ■ ■■■■ GridLayout.jsx ■■■■ ui/ ■ ■■■■ Button.jsx
■ ■■■■ Card.jsx ■ ■■■■ Badge.jsx ■ ■■■■ Modal.jsx ■ ■■■■ Tooltip.jsx ■ ■■■■
Spinner.jsx ■■■■ auth/ ■ ■■■■ LoginForm.jsx ■ ■■■■ RegisterForm.jsx ■■■■ layout/ ■
■■■ Topbar.jsx ■■■■ Sidebar.jsx ■■■■ ProtectedRoute.jsx ■■■■ pages/ ■■■■ LandingPage.jsx
■ ■■■■ Login.jsx ■■■■ Register.jsx ■■■■ Dashboard.jsx ■■■■ Settings.jsx ■■■■
Billing.jsx ■■■■ Docs.jsx ■■■■ hooks/ ■■■■ useWebSocket.js ■■■■ useRegime.js ■■■■
useSymbols.js ■■■■ useAuth.js ■■■■ useSettings.js ■■■■ store/ ■■■■ authStore.js ■■■■
symbolStore.js ■■■■ settingsStore.js ■■■■ services/ ■■■■ api.js ■■■■ websocket.js ■■■■
auth.js ■■■■ utils/ ■■■■ format.js ■■■■ constants.js ■■■■ validators.js ■■■■ App.jsx ■■■■
main.jsx
```

2. Routing Architecture

Route	Component	Auth Required	Notes
/	LandingPage	No	Public marketing page
/login	Login	No	Redirect to dashboard if already authed
/register	Register	No	Redirect to dashboard if already authed
/dashboard/:username	Dashboard	Yes	Main trading terminal
/dashboard/:username/settings	Settings	Yes	User config
/dashboard/:username/billing	Billing	Yes	Plan + payment
/docs	Docs	No	Public documentation
*	404	No	Redirect to /

Protected Route Pattern

Wrap all authenticated routes in a ProtectedRoute component that reads JWT from memory (not localStorage), validates expiry, and redirects to /login on failure. JWT is stored in memory only — never in localStorage or sessionStorage. Refresh token stored in httpOnly cookie set by the backend.

Security rule: JWT in memory. Refresh token in httpOnly cookie. Never localStorage. XSS cannot steal an httpOnly cookie.

3. Page: Landing

The landing page has one job: convert a semi-pro trader who arrived from Twitter/X or a Discord link into a trial signup. No fluff. No feature lists. Lead with the value proposition.

Layout — Top to Bottom

Section	Content	Height
Navbar	Logo left. Login + Start Free Trial buttons right.	64px
Hero	Large headline: 'Know what the market is doing before it does it.' Sub: One avoided bad trade covers 50 months of subscription. Two CTAs: Start Free Trial (primary), See Live Demo (secondary).	100vh
Live Preview	Embedded read-only dashboard widget showing real BTC regime state. Updates in real time. No auth required. Shows the product works.	500px
Three Points	1. Regime-aware — every signal knows what market it's in. 2. Explainable — no black box, full math shown. 3. Verifiable — dated public performance log.	300px
Pricing	Three cards: Trial (free), Standard (\$47/mo), Unlimited (\$97/mo). Each shows symbol limit, features, CTA.	400px
Performance Log	Public table of past regime calls with dates and outcomes. Builds trust. Updated automatically from database.	400px
Footer	Links: Docs, Twitter/X, Discord, Privacy, Terms.	80px

Performance

- Hero section — no JavaScript required to render. Static HTML + CSS only.
- Live preview widget — lazy loaded after hero is painted.
- Lighthouse score target: 95+ performance, 100 accessibility.

4. Page: Auth — Login + Register

Login

Field	Type	Validation
Email	email	Required, valid email format, max 255 chars
Password	password	Required, min 8 chars

- On submit: POST /api/auth/login. Receive access token (15min expiry) + set httpOnly refresh cookie.
- Store access token in Zustand authStore memory only — never persisted.
- On 401: clear token, redirect to /login.
- Rate limit feedback: show 'Too many attempts. Try again in Xs.' from API response.
- Forgot password link: POST /api/auth/forgot-password → email flow.

Register

Field	Type	Validation
Username	text	Required, 3-20 chars, alphanumeric + underscore, unique
Email	email	Required, valid format, unique
Password	password	Required, min 8 chars, 1 uppercase, 1 number
Confirm Password	password	Must match password

- On submit: POST /api/auth/register. Auto-login on success — receive token + redirect to dashboard.
- Username availability check: debounced GET /api/auth/check-username on input blur.
- Password strength indicator: visual bar, not text. Green = strong enough to submit.

Security

- CSRF token on all auth forms — fetched from GET /api/auth/csrf before form render.
- All inputs sanitised client-side before submission. Server validates independently.
- No error messages that reveal whether email exists — always generic on failure.
- Auto-redirect to intended page after login (store intended route in session storage, not URL).

5. Page: Dashboard — Trading Terminal

The dashboard is the core product. Every design decision optimises for information density and zero latency between data arrival and screen update. Bloomberg aesthetic — dark, mono font, data first.

Layout Structure — Top to Bottom

Zone	Height	Content
Topbar	42px	Logo left. Symbol tabs centre. Live status + timestamp + username right.
Summary Strip	32px	Full-width single line. LLM-generated. Updates every 30s. e.g. 'BTC trending regime, 87% confidence, bullish confluence 0.74 — on-chain outflow spike confirms institutional accumulation.'
Alert Strip	28px	Hidden when no alerts. Flashes amber on regime transition >75% conf. Flashes red on VPIN >90th percentile. Auto-dismisses after 10s.
Symbol Tabs	36px	One tab per active symbol. Active tab highlighted. + button to add symbol (up to plan limit).
Tab Content	fill	4 sub-tabs per symbol: Microstructure / Regime / Alt Data / Causal AI.
Status Bar	26px	Funding rate, OI, liquidations 1h, mempool fees, model info.

Symbol Tabs

- Each symbol tab shows a coloured dot: green (trending), blue (mean-reverting), yellow (volatile), red (illiquid).
- Clicking + opens a modal to search and add a symbol. Validates against cached Binance symbol list.
- Right-click or long-press on a tab opens: Remove Symbol / Symbol Settings.
- Tab order is draggable — user preference persisted to user_settings.

Sub-Tab 1: Microstructure

Widget	Content
Order Book	Live L2 heatmap. Bid levels green, ask levels red. Depth visualised as horizontal bars behind price. OFI imbalance strip at bottom with directional indicator.
Metrics Grid	6 cells: VPIN (colour-coded by toxicity level), Kyle Lambda, CVD (running), Trade Intensity, Adverse Selection %, VWAP Deviation.
OFI Chart	Rolling 200-tick line chart of OFI with mid-price overlay. Regime-shaded background (colour changes with regime state).
Spread Decomp	Stacked bar: adverse selection component vs inventory cost component. Updates on each tick.

Sub-Tab 2: Regime

Widget	Content
Current State	Large regime name + confidence %. Colour matches regime type. Time in current regime. Transition probability warning if >20%.

Probability Bars	4 horizontal bars showing posterior probability for each regime. Animates on update.
Transition Matrix	4x4 grid showing learned transition probabilities. Heatmap coloured by probability value.
Regime History	Timeline of past 24h regime states with durations. Colour-coded bands on a horizontal bar.
Model Info	Tier (dedicated/high/mid/low), trained at, n_observations, reliability status.

Sub-Tab 3: Alt Data Intelligence

Dashboard inside a dashboard. Full sub-layout with its own widget grid.

Widget	Content
Confluence Score	Large score -1.0 to +1.0 with direction label. Animated arc gauge. Threshold lines at +0.7 and -0.7.
Per-Source Cards	One card per source: Reddit, On-chain, Google Trends, Macro FRED. Each shows: current score, z-score vs baseline, trend arrow, last updated.
Correlation Matrix	Cross-source correlation heatmap. Shows which sources move together.
Lead/Lag Chart	Line chart showing each source vs price with configurable lag offset. Visual answer to: does this source lead or follow price?
Macro Panel	FRED series display: yield curve, CPI trend, fed funds rate. Risk-on / risk-off classification with confidence.

Sub-Tab 4: Causal AI

Widget	Content
Insight Feed	Scrollable list of LLM causal insights. Each card shows: source (SEC/News/Transcript), timestamp, causal summary, confidence tag, affected assets.
Chat Interface	Full chat UI. User types a question. Backend sends question + last N microstructure + regime + altdata snapshots as RAG context to OpenRouter. Response streams via SSE. Chat history persisted to session storage, not database.
Context Panel	Sidebar showing exactly what context was sent with the last query. Regime state, VPIN, confluence score, last 3 insights. Transparency — user sees what AI sees.

Widget Grid — react-grid-layout

- Each sub-tab has its own draggable grid. Layout persisted to user_settings on drag-end.
- Default layout loaded from user_settings on mount. Falls back to hardcoded default if none saved.
- Widgets have a minimum size. Cannot be resized below minimum that makes content unreadable.
- Drag handle is the widget header bar only — not the entire widget body.
- Layout changes debounced 500ms before saving to avoid hammering the API.

6. Page: Settings

Route: `/dashboard/:username/settings`. Full-page settings. Not a modal. Sidebar navigation between setting categories.

Section	Settings
Symbols	Add / remove symbols. Shows plan limit and current count. Warns on low-data symbols before adding.
Alerts	Discord webhook URL (tested on save). Confluence threshold slider (0.5 — 0.95). Regime confidence threshold for alerts.
Alt Data	Reddit: subreddits list (add/remove). Google Trends: keywords list. FRED: series selection from dropdown of available series.
Dashboard	Default tab on load (Microstructure/Regime/AltData/CausalAI). Reset layout button (restores default widget positions). Display timezone selector.
Account	Change email (requires current password). Change password (requires current password). Delete account (requires typing 'DELETE' to confirm).
Notifications	Email alerts on/off. Regime transition alerts on/off. Weekly performance summary on/off.

Save Behaviour

- Each section has its own Save button. Not a global save. Reduces accidental overwrites.
- Unsaved changes indicator — dot on section tab if changes not yet saved.
- On save: PATCH `/api/users/:username/settings` with only changed fields.
- Discord webhook: test button sends a real test message before saving.

7. Page: Billing

Route: /dashboard/:username/billing. Shows current plan, usage, and upgrade/downgrade options.

Current Plan Card

- Plan name, price, renewal date, symbol count used vs limit.
- Status badge: Active / Trial (X days remaining) / Grace Period / Expired.
- Cancel subscription button — requires confirmation modal. Explains what happens on cancel.

Plan Comparison

Feature	Trial	Standard \$47/mo	Unlimited \$97/mo
Symbols	3	10	Unlimited
Regime detection	Yes	Yes	Yes
Microstructure	Yes	Yes	Yes
Alt Data	Yes	Yes	Yes
Causal AI chat	5 queries/day	50 queries/day	Unlimited
Discord alerts	Yes	Yes	Yes
API access	No	No	Yes
Trial duration	14 days	—	—

Payment

- Crypto payments via OxaPay. No card required.
- On upgrade click: POST /api/payment/create → receive OxaPay invoice URL → open in new tab.
- Page polls GET /api/payment/status/:id every 5 seconds while invoice tab is open.
- On Paid status: refresh plan display, show success toast.
- Payment history table: date, amount, currency, status, invoice ID.

Grace Period Handling

- If payment fails: show banner 'Payment failed — your access continues for 3 days.'
- Banner persists on all pages until resolved. Cannot be dismissed.
- On expiry: redirect to /dashboard/:username/billing with 'Your trial has ended' message.

8. Page: Docs

Route: /docs. Public — no auth required. Static content, fast load, searchable.

Structure

Section	Content
Getting Started	Account creation, first symbol, understanding the dashboard.
Regime Detection	What the four regimes mean. How HMM works in plain English. How to use regime state in trading decisions.
Microstructure	What VPIN measures. What OFI tells you. How to read the order book heatmap.
Alt Data	What confluence score means. How each source is weighted. Lead vs lag classification explained.
Causal AI	How the RAG context works. What to ask. Limitations — not financial advice.
Alerts	Setting up Discord webhook. Configuring thresholds.
Symbol Management	Adding symbols. Data sufficiency warning explained. Tier model fallback explained.
Performance Log	How signal accuracy is tracked. Walk-forward validation explained.
FAQ	Common questions. Known limitations. Pricing.

Implementation

- Markdown files in /docs directory, rendered via a lightweight MD parser.
- Left sidebar navigation. Active section highlighted. Sticky on scroll.
- Search via Ctrl+K — client-side search over all doc content.
- Copy button on all code blocks.

9. WebSocket Integration

Single WebSocket connection per authenticated user. Multiplexed — all symbols over one connection. Reconnects automatically on disconnect.

Connection Lifecycle

Event	Action
Mount	Open WS to wss://vektorlabs.xyz/ws. Send auth message with JWT.
Auth success	Server confirms. Subscribe to active symbols automatically.
Symbol added	Send subscribe:{symbol} message. Server starts pushing that symbol.
Symbol removed	Send unsubscribe:{symbol} message.
Disconnect	Reconnect with exponential backoff: 1s, 2s, 4s, 8s, max 30s. Show 'Reconnecting...' in status bar. Never show error to user during retry.
JWT expiry	useAuth hook detects 401 on reconnect. Silently refresh via httpOnly cookie. Reconnect with new token. User sees nothing.
Unmount	Close connection cleanly. Cancel all subscriptions.

Message Types — Inbound

Type	Payload	Consumer
regime_update	symbol, regime, confidence, transition_probs, time_in_regime	RegimeWidget, SymbolTab dot
microstructure_update	symbol, ofi, vpin, cvd, spread, kyle_lambda, trade_intensity	MicrostructureWidget
altdata_signal	source, value, z_score, timestamp	AltDataWidget
altdata_confluence	symbol, score, direction, per_source	AltDataWidget, AlertStrip
llm_insight	source, summary, confidence, assets, timestamp	LLMInsightWidget
alert	type, symbol, message, severity	AlertStrip, toast
summary_update	text, timestamp	SummaryStrip
orderbook_update	symbol, bids, asks, spread, ofi_imbalance	OrderBookWidget

useWebSocket Hook

- Singleton — one instance shared across all components via Zustand store.
- Components subscribe to specific message types. Do not pass raw WS messages to components.
- Message dispatch: incoming message → parse type → route to correct Zustand slice → components re-render.
- Heartbeat: send ping every 30s. If no pong in 5s, assume dead, reconnect.

10. Component Specifications

Design System

- Font: IBM Plex Mono for all data values. IBM Plex Sans for prose and labels.
- Colors: accent green #00ff88, cyan #00ccff, warn orange #ff6b35, red #ff4455, yellow #ff5c542.
- Background: #0b0b1a base, #11112a cards, #16162e card alt row.
- All interactive elements have :focus-visible ring for keyboard navigation.
- No transitions longer than 150ms on data-bearing elements — latency perception.

Regime Badge

Regime	Color	Behaviour
TRENDING	Green	Solid fill
MEAN-REVERTING	Cyan	Solid fill
VOLATILE	Yellow	Pulsing animation
ILLIQUID	Red	Pulsing animation + warning icon

VPIN Cell

- 0.0 — 0.3: green — low toxicity
- 0.3 — 0.6: yellow — moderate toxicity
- 0.6 — 1.0: red — high toxicity + subtle background highlight

Order Book Heatmap

- Price levels rendered as rows. Depth visualised as horizontal bar behind price using opacity.
- Best bid / ask row slightly larger font and brighter color.
- New levels animate in. Removed levels fade out (150ms).
- Large orders (>2x average) get a whale icon and brighter background.
- Maximum 20 levels each side rendered — more causes layout thrash.

SummaryStrip

- Typewriter animation on text update — 30 chars/second.
- Background subtly shifts color based on overall market sentiment direction.
- Timestamp shown on hover.

AlertStrip

- Slides down from above symbol tabs. 28px height.
- Amber background for regime alerts. Red for toxicity alerts.
- Auto-dismisses after 10 seconds. User can dismiss manually.
- Multiple alerts queue — shown one at a time.

11. State Management

Three Zustand stores. React Query for all server state. No Redux — too much boilerplate for this scale.

Store	Contents	Persisted
authStore	accessToken (memory), username, plan, expiry	No — memory only
symbolStore	activeSymbols[], currentSymbol, regimeStates{}, microstructureData{}, altdataData{}, llmInsights[]	No — WebSocket feeds it
settingsStore	userSettings{}, layoutConfigs{symbol: layout}	No — fetched from API on mount

React Query Usage

- GET /api/users/:username/settings → useQuery, refetch on window focus.
- PATCH settings → useMutation with optimistic update.
- GET /api/regime/current → useQuery with 0 staleTime (WebSocket handles freshness).
- All mutations invalidate relevant queries on success.

Data Flow

WebSocket message → parse → symbolStore.updateRegime(symbol, data) → RegimeWidget re-renders. No prop drilling. Components subscribe directly to store slices they need.

12. Security

Security is non-negotiable. Every item in this section is a hard requirement, not a suggestion.

Authentication

- Access token: JWT, 15-minute expiry, stored in memory (Zustand) only.
- Refresh token: httpOnly, Secure, SameSite=Strict cookie. 7-day expiry.
- On access token expiry: silent refresh via POST /api/auth/refresh using cookie.
- On refresh failure (cookie expired/revoked): redirect to /login. Clear all state.
- Never store any token in localStorage, sessionStorage, or any persistent browser storage.

XSS Prevention

- React escapes all rendered content by default. Never use dangerouslySetInnerHTML.
- All LLM-generated text rendered as plain text — never as HTML.
- Content Security Policy header set by Nginx — disallows inline scripts.
- All external URLs opened with rel='noopener noreferrer'.

CSRF Prevention

- All state-mutating requests (POST, PATCH, DELETE) include CSRF token in X-CSRF-Token header.
- CSRF token fetched from GET /api/auth/csrf on app mount. Stored in memory.
- Backend validates CSRF token on every mutating request.

Input Validation

- All form inputs validated client-side before submission. Server validates independently — always.
- Symbol input validated against cached Binance symbol list before sending to server.
- Discord webhook URL validated format client-side. Server sends test message to validate it works.
- Max length enforced on all text inputs to prevent oversized payloads.

WebSocket Security

- First message must be auth with valid JWT. Connection closed immediately if auth fails.
- All subsequent messages ignored until auth confirmed.
- Connection rate limited per IP at Nginx level.
- Reconnect uses fresh JWT from authStore — never caches connection credentials.

Dependency Security

- npm audit run in CI on every push. Block deploy on high severity vulnerabilities.
- Dependabot enabled for automated security updates.
- No CDN-hosted dependencies — all bundled. Removes supply chain attack surface.

13. Performance

Bundle

- Code split by route. Each page loads only its own bundle.
- react-grid-layout loaded only on Dashboard route.
- recharts loaded only on pages that use charts.
- Target: initial JS bundle < 150KB gzipped.

Rendering

- Order book widget: virtualised list — only render visible rows. Never render all 50 levels if only 20 are visible.
- WebSocket message handler: batch DOM updates with requestAnimationFrame. Never update DOM synchronously on every tick.
- Charts: update at maximum 10fps. Throttle incoming data updates.
- Zustand selectors: components subscribe to specific slice, not entire store. Prevents unnecessary re-renders.

Network

- All API responses cached by React Query. Stale-while-revalidate pattern.
- Images: WebP format, lazy loaded, explicit width/height to prevent layout shift.
- Fonts: preloaded in . Subset to only used characters for mono font.
- Nginx gzip compression on all text assets.

Targets

Metric	Target
First Contentful Paint	< 1.2s
Time to Interactive	< 2.5s
Largest Contentful Paint	< 2.0s
Cumulative Layout Shift	< 0.05
WebSocket message to paint	< 16ms (one frame at 60fps)

14. Accessibility

- WCAG 2.1 AA compliance minimum. Target AAA where feasible.
- All interactive elements reachable via keyboard. Tab order logical.
- Focus visible ring on all focusable elements — never removed with `outline:none` without replacement.
- All images have meaningful alt text. Decorative images have empty alt.
- Color is never the only indicator. Regime states have text labels + icons in addition to color.
- Minimum contrast ratio 4.5:1 for normal text, 3:1 for large text.
- All form inputs have associated elements.
- Error messages associated with inputs via `aria-describedby`.
- Live data regions marked with `aria-live='polite'` — screen readers announce updates without interrupting.
- Alert strip uses `aria-live='assertive'` for high-severity alerts.
- Modal dialogs trap focus. Escape key closes. Focus returns to trigger on close.
- All charts have text alternative (data table) accessible via a visually-hidden element.

15. Error Handling

API Errors

Status	Behaviour
400	Show field-level validation errors from response body.
401	Attempt silent token refresh. If fails, redirect to /login.
403	Show 'You do not have permission' toast. Do not redirect.
404	Show inline 'Not found' message in relevant component. Not a full page redirect.
422	Show validation errors from response body on relevant form fields.
429	Show 'Too many requests. Try again in Xs.' with countdown from Retry-After header.
500	Show 'Something went wrong. We have been notified.' Do not show error details.
503	Show 'System maintenance in progress.' with retry button.

WebSocket Errors

- Disconnect: silent reconnect with backoff. Show subtle 'Reconnecting...' in status bar only.
- Auth failure: redirect to /login with 'Session expired' message.
- Malformed message: log to console in dev, silently discard in production.
- Never show raw error objects or stack traces to users in production.

React Error Boundary

- Each widget wrapped in ErrorBoundary. If widget crashes, show 'Widget unavailable' placeholder.
- Dashboard as a whole has a top-level ErrorBoundary — page-level crash shows reload button.
- Errors logged to console in dev. In production, logged to server via POST /api/errors (fire-and-forget).

16. Testing Strategy

Type	Tool	Coverage Target	What to Test
Unit	Vitest	80%	Utility functions, formatters, validators, Zustand store actions.
Component	React Testing Library	Key components	Auth forms, regime badge, VPIN cell, alert strip, symbol tab.
Integration	Vitest + MSW	Critical paths	Login flow, symbol add flow, settings save, WebSocket message handling.
E2E	Playwright	Happy paths	Register → dashboard → add symbol → receive regime update → change settings.
Accessibility	axe-core	All pages	Run on every page in CI. Zero violations on A and AA rules.

CI Pipeline

- On every push: lint (ESLint), type check (if using TypeScript), unit tests, component tests.
- On PR to main: full test suite including E2E. Block merge on failure.
- npm audit on every push. Block on high severity.

17. Deployment

Build

npm run build → dist/ directory. Static files only. Served by Nginx.

Nginx Config — Key Points

- Serve dist/ for all routes except /api and /ws — SPA routing requires this.
- Cache static assets (JS, CSS, fonts) with 1-year max-age. Content-hash in filenames ensures cache busting.
- Cache HTML with no-store — always fresh.
- HTTPS only — HTTP redirects to HTTPS.
- HSTS header: max-age=31536000; includeSubDomains.
- Content-Security-Policy: default-src 'self'. Disallow inline scripts and eval.
- X-Frame-Options: DENY.
- X-Content-Type-Options: nosniff.
- Referrer-Policy: strict-origin-when-cross-origin.

CI/CD

- GitHub Actions: on push to main → npm ci → npm run build → rsync dist/ to VPS → Nginx reload.
- No Docker for frontend — static files served directly by Nginx. Simpler, faster.
- Rollback: keep last 3 builds on VPS. Rollback script swaps symlink in seconds.

Environment Variables

- VITE_API_URL=https://vektorlabs.xyz
- VITE_WS_URL=wss://vektorlabs.xyz/ws
- No secrets in frontend env vars — they are visible in the bundle. API keys stay server-side.

Frontend deployment cost: \$0. Static files served by same Nginx instance as the API. No separate hosting needed.